

SATELINK DePIN NETWORK

PRODUCTION READINESS AUDIT REPORT

Date	2026-03-16
Auditor	Distributed Systems Architecture Review
Repository	satelink-mvp (branch: claude/interesting-herschel)
Scope	Full end-to-end infrastructure audit - 18 parts
Objective	Determine launch readiness for 72-hour autonomous stability test and real network launch

READINESS SCORES SUMMARY

Metric	Score	Rating
Runtime Stability	52%	FAILING
Economic Pipeline	35%	CRITICAL
Production Readiness	28%	NOT READY
72-Hour Test Readiness	22%	BLOCKED
Real Launch Readiness	15%	BLOCKED

VERDICT: NOT READY for production or 72-hour stability test.
Requires 5-stage fix blueprint (estimated 80-120 engineering hours).

TABLE OF CONTENTS

Part 1 - System Architecture Audit
Part 2 - Multi-RPC Support Verification
Part 3 - Wallet Login and Auth Flow
Part 4 - UI/UX Flow Validation
Part 5 - Environment Configuration Audit
Part 6 - Connector System Audit
Part 7 - Economic Pipeline Verification
Part 8 - Node Network Audit
Part 9 - Scheduler and Runtime Loop Review
Part 10 - Database Integrity
Part 11 - Monitoring Stack
Part 12 - Docker Infrastructure
Part 13 - Security Review
Part 14 - 72-Hour Test Readiness
Part 15 - Real Network Launch Readiness
Part 16 - Infrastructure Launch Checklist
Part 17 - Fix Blueprint
Part 18 - Final Output & Scores

PART 1 — SYSTEM ARCHITECTURE AUDIT

Architecture Module Inventory

Module	Location	Status	Impl Quality
API Backend	apps/api/server.js → Express app factory	WORKING	Production-grade
Operations Engine	src/core/operations_engine.js (629 LOC)	HARDENED	Real revenue tracking, double-entry ledger
Job Scheduler	src/scheduler/job_scheduler.js	WORKING	Redis Streams, priority queues
Node Worker	apps/api/worker.js	PARTIAL	Missing error recovery, graceful shutdown
Economic Ledger	src/economics/ (39+ files)	WORKING	Double-entry, tamper-evident hash chain
Epoch Pipeline	src/economics/epoch_aggregator.js	CODED, NOT WIRED	closeEpoch() never called
Settlement Engine	src/settlement/settlement_engine.js	CODED, NOT WIRED	Full state machine, never invoked
RPC Gateway	src/gateway/routes/rpc.js	WORKING	Chain routing with fallback providers
Demand Connectors	src/workloads/connectors/ (6 files)	ALL STUBS	Generate synthetic demand only
Monitoring Stack	src/monitoring/ (25+ files)	PARTIAL	Prometheus metrics exist, watchdog is health-check only
Dashboard Frontend	apps/dashboard/ (Next.js 16)	85% COMPLETE	Test-mode auth, most pages functional
Docker Infrastructure	Root + infra/docker/	60% READY	Missing health checks, hardcoded secrets

Wiring Verification

```
CLIENT REQUEST
→ Express Gateway (routes.js) ..... ■ WORKING
→ Auth Middleware (auth_middleware.js) ..... ■ HARDENED
→ Operations Engine (executeOp) ..... ■ WORKING
→ Revenue Event (revenue_events_v2) ... ■ WORKING
→ Ledger Entry (economic_ledger) ... ■ WORKING
→ Epoch Aggregation ..... ■ NEVER CALLED
→ Batch Creation ..... ■ MISSING CODE
→ Settlement Engine ..... ■ NOT WIRED
→ EVM Adapter ..... ■ NOT REGISTERED
→ USDT Transfer ..... ■ NOT HAPPENING
```

Critical Finding: The first half of the pipeline (request → revenue event → ledger) works perfectly. The second half (epoch close → batch → settle → pay) is coded but not wired together.

PART 2 — MULTI-RPC SUPPORT VERIFICATION

Architecture Assessment

Component	Status	Details
RPC Gateway Route	REAL	POST /rpc/:chain accepts JSON-RPC payloads
Chain Adapters	REAL	Ethereum, Polygon, Arbitrum, Fuse, Solana, Base
ExecutionAssuranceRouter	REAL	Priority routing: community → genesis → external
Provider Fallback	MOCK	provider_fallback_adapter.js returns simulated responses
Load Balancing	NOT IMPLEMENTED	No round-robin or weighted routing between providers
RPC_DEFAULT_TARGET	NOT USED	No env var controlling default chain target
External Providers	PLACEHOLDER	Infura/Alchemy/QuickNode endpoints use SATELINK_INTERNAL keys

Supported Chains

Chain	Adapter	Validation	Execution	Real Provider
Ethereum	ethereum.js	■ JSON-RPC	MOCK	Infura (placeholder key)
Polygon	polygon.js	■ JSON-RPC	MOCK	Infura (placeholder key)
Arbitrum	arbitrum.js	■ JSON-RPC	MOCK	Alchemy (placeholder key)
Fuse	fuse.js	■ JSON-RPC	MOCK	Native endpoint
Solana	solana.js	■ Solana-specific	MOCK	QuickNode (placeholder key)
Base	Listed in source	Minimal	MOCK	None configured

Multi-RPC Verdict

The system supports multiple chains structurally but all execution is **MOCK**. The **ExecutionAssuranceRouter** correctly routes through internal nodes first, then external providers, but the **ProviderFallbackAdapter** simulates 150ms latency and returns fake responses.

Blueprint for Real Multi-RPC

- Stage 1:** Replace **ProviderFallbackAdapter** with real HTTP forwarding to external RPC endpoints
- Stage 2:** Configure real API keys for Infura/Alchemy/QuickNode via environment variables
- Stage 3:** Add health-check probing for external providers (latency + error rate tracking)
- Stage 4:** Implement weighted round-robin load balancing across healthy providers
- Stage 5:** Add circuit breaker per provider (fail-open to next provider)

PART 3 — WALLET LOGIN AND AUTH FLOW

Current State

Component	Status	Details
Login Page	TEST MODE ONLY	Hardcoded test buttons for admin, node_operator, builder, distributor, enterprise
Test Endpoint	/__test/auth/login	Bypasses wallet verification entirely
JWT Auth Flow	COMPLETE	Token → localStorage → Authorization header → 401 redirect
Token Refresh	IMPLEMENTED	apiClient.ts has silent re-auth via embedded wallet signing
Embedded Wallet	IMPLEMENTED	Device-bound, WebCrypto (AES-GCM) encrypted, IndexedDB stored
Signing Guard	IMPLEMENTED	Modal popup intercepts satelink:sign_request events
MetaMask/WalletConnect	NOT IMPLEMENTED	No wagmi, no Web3Modal, no external wallet integration
Backend Auth	HARDENED	No JWT_SECRET fallback, HS256 enforced, 32+ char requirement

Auth Flow Trace

CURRENT (Test Mode):
Click test button → POST /__test/auth/login → JWT returned → dashboard loads

INTENDED (Production):
Connect wallet → Sign nonce message → POST /auth/embedded/finish → JWT returned → dashboard loads

MISSING:

- ✗ No MetaMask popup or WalletConnect modal
- ✗ No wallet address displayed on login page
- ✗ No nonce challenge from backend (GET /auth/embedded/start exists but not called from login)
- ✗ Login page has NO wallet connect UI

Auth Repair Blueprint

1. Install **wagmi** + **@rainbow-me/rainbowkit** (or **@web3modal/react**)
2. Add WalletConnectButton component to login page
3. On connect: call **GET /auth/embedded/start** to get nonce
4. Sign nonce with connected wallet
5. Submit signature to **POST /auth/embedded/finish**
6. Store JWT, redirect to role-based dashboard

PART 4 — UI/UX FLOW VALIDATION

Page Audit Results

Page	Route	Data Source	Status
Landing	/	useNetworkStats() API	■ Real data + marketing
Login	/login	Test endpoint	■■ Test mode only
Node Dashboard	/node	/node/stats + SSE stream	■ 90% complete
Node Earnings	/node/earnings	/node/me/earnings	■ API connected
Node Claim	/node/claim	/node/me/claim	■■ Not fully wired
Builder Dashboard	/builder	/builder-api/usage + keys	■ Fully implemented
Builder Keys	/builder/keys	/builder-api/keys CRUD	■ Working
Admin Dashboard	/admin	Redirects to command center	■ Navigation works
Admin (70+ pages)	/admin/*	Various admin APIs	■■ Pages exist, data varies
Network	/network	useNetworkStats()	■■ Real metrics + mock node cards
Enterprise	/enterprise	Enterprise APIs	■■ Exists, not verified
Distributor	/distributor	Distributor APIs	■■ Exists, not verified
Settlement	/settlement	Settlement mode API	■ Shows SIMULATED/SHADOW/EVM
Status	/status	Health endpoints	■ Connected

UI Issues Found

1. **Network Page:** AnimatedNodeCards use random IDs and mock statuses (18 fake nodes)
2. **Builder Dashboard:** "System Health" card shows hardcoded "OPTIMAL" with dummy latency
3. **Node Claim/Withdraw:** Buttons exist but settlement pipeline is broken (claims created, never paid)
4. **Missing Error Boundaries:** No React error boundaries — component crashes show blank screens
5. **Dual Auth Hooks:** Both `use-auth.tsx` and `context/AuthContext.tsx` exist with different implementations
6. ****Missing `useUserMode` hook:**** Imported in SigningGuard but doesn't exist

UI Framework

- **Styling:** Tailwind CSS v4 + shadcn/ui components
- **Charts:** Recharts for earnings visualization
- **Animations:** Framer Motion
- **Data Fetching:** SWR (stale-while-revalidate, 30-60s refresh)
- **Forms:** react-hook-form + zod validation
- **Icons:** lucide-react
- **Notifications:** sonner (toast)

PART 5 — ENVIRONMENT CONFIGURATION AUDIT

Required Variables Status

Variable	.env.example	.env.staging.example	Required For	Status
JWT_SECRET	dev_secret_keys_only_for_local_use	change_me_to_something_secure	Auth	■■ Placeholder
JWT_REFRESH_SECRET	NOT LISTED	NOT LISTED	Token refresh	■ MISSING, has hardcoded fallback
PASSWORD_SALT	NOT LISTED	NOT LISTED	Password hashing	■ MISSING, cascades to JWT_SECRET
IP_HASH_SALT	dev_salt_1	NOT LISTED	Privacy	■■ Dev value
ADMIN_API_KEY	NOT LISTED	legacy_key_fallback	Admin access	■ MISSING from main example
DATABASE_URL	PostgreSQL string	Commented out	Database	■■ Not used (SQLite active)
REDIS_URL	redis://localhost:6379	NOT LISTED	Queue	■ Present
SQLITE_PATH	NOT LISTED	NOT LISTED	SQLite DB	Uses default satelink.db
USDT_CONTRACT_ADDRESS	NOT LISTED	NOT LISTED	Settlement	■ MISSING
TREASURY_ADDRESS	NOT LISTED	NOT LISTED	Revenue split	■ MISSING
NODE_REGISTRY_ADDRESS	NOT LISTED	NOT LISTED	Node mgmt	■ MISSING
CLAIMS_CONTRACT_ADDRESS	NOT LISTED	NOT LISTED	Claims	■ MISSING
SPLIT_ENGINE_ADDRESS	NOT LISTED	NOT LISTED	Revenue split	■ MISSING
RPC_DEFAULT_TARGET	NOT LISTED	NOT LISTED	RPC routing	■ MISSING
SETTLEMENT_EVM_ENABLED	NOT LISTED	NOT LISTED	Real settlement	■ MISSING
SETTLEMENT_EVM_CHAIN_NAME	NOT LISTED	NOT LISTED	Chain selection	■ MISSING
SETTLEMENT_EVM_SIGNER_PRIVATE_KEY	NOT LISTED	NOT LISTED	Tx signing	■ MISSING
SETTLEMENT_EVM_TOKEN_MAP_JSON	NOT LISTED	NOT LISTED	Token config	■ MISSING
FEATURE_REAL_SETTLEMENT	NOT LISTED	NOT LISTED	Settlement mode	■ MISSING
CORS_ORIGINS	NOT LISTED	NOT LISTED	Security	■ MISSING (allows all when empty)
FLYWHEEL_ENABLED	NOT LISTED	NOT LISTED	Demand gen	Defaults to true

Security Risks

- **14 critical environment variables** not documented in .env.example
- **JWT_REFRESH_SECRET** has hardcoded fallback `'satelink-fallback-refresh-token'`
- **PASSWORD_SALT** cascades: `PASSWORD_SALT` → `JWT_SECRET` → `'satelink_fallback_salt'`
- **IP_HASH_SALT** defaults: `'satelink_default_salt_change_me'` in some routes
- **CORS** allows all origins when `CORS_ORIGINS` is empty

PART 6 — CONNECTOR SYSTEM AUDIT

Connector Status Matrix

Connector	File	Type	Status	External API
RPC Market	rpc_market_connector.js	Discovery	STUB	None — generates 0-4 fake eth_* jobs
AI Market	ai_market_connector.js	Discovery	STUB	None — generates 0-2 fake AI jobs
Automation Market	automation_market_connector.js	Discovery	STUB	None — generates 0-2 fake cron jobs
Oracle Monitoring	oracle_monitoring_connector.js	Discovery	STUB	None — generates 1-2 fake oracle jobs
Overflow Compute	overflow_compute_connector.js	Discovery	STUB	None — generates 0-1 fake batch jobs
Indexing	indexing_connector.js	Discovery	STUB	None — generates 0-1 fake indexing jobs

Workload Source Status

Source	File	Status	Behavior
RPC Source	rpc_source.js	STUB	Generates random RPC workloads with hardcoded chains
AI Source	ai_source.js	STUB	Generates random inference requests
Cron Source	cron_source.js	STUB	Generates scheduled automation triggers
Webhook Source	webhook_source.js	STUB	Generates mock webhook deliveries to example.com

Adapter Layer (Real)

Adapter	Status	Validated Methods/Types
RPC Adapter	REAL	eth_call, eth_blockNumber, eth_getBalance, eth_sendRawTransaction + 5 more
AI Adapter	REAL	openai, anthropic, gemini, llama, mistral, stable-diffusion, whisper, embedding
Automation Adapter	REAL	cron, condition, event_driven, manual, chain_event triggers
Webhook Adapter	REAL	Event + data validation, 10KB payload limit

Feature Flags for Connectors

No explicit `ENABLE_RPC_CONNECTOR`, `ENABLE_AI_CONNECTOR`, etc. flags found. Instead:

- `FLAG_DISABLE_RPC` — disables entire RPC route
- `FLYWHEEL_ENABLED` — controls demand flywheel engine
- Feature flags are DB-driven (modes: OFF, ON, WHITELIST, PERCENT)

Connector Recommendation for Launch

Minimum connectors for 72-hour test: RPC Market Connector must be converted from stub to real (connecting to actual RPC demand sources or at minimum generating realistic traffic patterns).

For real launch: At least 2 connectors must produce real market demand (RPC + one of AI/Automation).

PART 7 — ECONOMIC PIPELINE VERIFICATION

Pipeline Stage Completion

STAGE 1: API Request → Revenue Event ■ FULLY IMPLEMENTED

- Operations Engine records to revenue_events_v2
- Dynamic surge pricing applied
- Idempotency key prevents double-charging
- Double-entry ledger recorded (CLIENT debit ↔ PLATFORM credit)

STAGE 2: Revenue Events → Epoch Aggregation ■ CODED, ■ NEVER CALLED

- epoch_aggregator.js:closeEpoch() is perfect
- Atomic transaction: SUM revenue → apply 50/30/20 split → distribute by ops weight
- FuturesEscrow integration deducts investor obligations
- BUT: No endpoint or cron job calls closeEpoch()

STAGE 3: Epoch Earnings → Claim ■ WORKING

- ECDSA signature verification on claim
- Atomic: all unclaimed epochs → CLAIMED status
- Ledger: REWARDS_PAYABLE → PAYOUTS_PAYABLE

STAGE 4: Claim → Withdrawal Record ■ WORKING

- Withdrawal entry created in 'PENDING' status
- Balance validation enforced

STAGE 5: Withdrawal → Payout Batch ■ MISSING CODE

- NO code converts withdrawals to payout_batches_v2
- This is a GAP – withdrawals orphaned forever

STAGE 6: Batch → Settlement Engine ■ NOT WIRED

- SettlementEngine.processQueue() exists but is NEVER CALLED
- No cron job, no timer, no endpoint invokes it

STAGE 7: Settlement → On-Chain Transfer ■ NOT REGISTERED

- EvmAdapter fully coded (nonce management, ERC20 transfers)
- But adapter_registry has no adapters registered
- Defaults to SIMULATED adapter (instant mock completion)

STAGE 8: USDT Receipt ■ NOT HAPPENING

- Pipeline terminates at withdrawal record creation

50/30/20 Split Verification

Verified in 3 locations:

Location	Node Pool	Platform	Distribution	Status
epoch_aggregator.js	totalRevenue * 0.5	totalRevenue * 0.3	totalRevenue * 0.2	■ Correct
SplitEngine.sol	5000 BPS	3000 BPS	2000 BPS	■ Correct
RevenueDistributor.sol	5000 BPS	3000 BPS	2000 BPS	■ Correct

Platform Sub-Split (of the 30%):

- Core Treasury: 70% of platform share (= 21% total)
- Builder Rewards: 30% of platform share (= 9% total)

Infrastructure Reserve: 10% of node pool for managed nodes (= 5% total for managed)

Smart Contracts Audit

Contract	Lines	Purpose	Status
SplitEngine.sol	~120	Governance-controlled split config	■ Audit-ready
RevenueDistributor.sol	~180	USDT distribution with SafeERC20	■ Fixed (ETH→USDT), reentrancy guarded
ClaimsWithdrawals.sol	~200	Merkle-proof claims + partial withdrawals	■ OpenZeppelin standard double-hash
ClaimsContract.sol	~150	Legacy claims	■ Working
RevenueVault.sol	~100	USDT liquidity vault	■ Access-controlled
EpochAnchor.sol	~80	On-chain epoch root storage	■ Working
NodeRegistryV2.sol	~150	Node registration	■ Working
EligibilityPolicy.sol	~60	Node eligibility rules	■ Working
MockUSDT.sol	~30	Test token	N/A (test only)

PART 8 — NODE NETWORK AUDIT**Node Lifecycle Verification**

Phase	Component	Status
Registration	node_registry.js	■ Working — wallet-based registration
Pairing	lifecycle_manager.js	■■ BROKEN — DB API mismatch (uses .query().get() vs .prepare())
Heartbeat	node_heartbeat.js	■ Working — receives CPU/memory/capacity/latency
Reputation	node_reputation.js	■ Working — composite score: (uptime×0.4 + success×0.4 + latency×0.2)
Job Assignment	job_matching_engine.js	■ Working — matches jobs to capable nodes
Job Execution	ExecutionAssuranceRouter	■■ MOCK — simulated execution, not real
Reward Accounting	operations_engine.js	■ Working — records revenue per node per op

Heartbeat System

- **Receives:** { node_id, cpu_usage, memory_usage, capacity_available, latency_ms }
- **Updates:** Node capacity via **NodeCapacity** manager
- **Triggers:** Reputation recalculation (if enabled)
- **Issue:** Stats accumulate **in-memory** — lost on restart

Reputation Scoring

Score = (uptime_pct × 0.4) + (job_success_rate × 0.4) + (latency_score × 0.2)

Latency mapping: ≤50ms = 100, ≥2000ms = 0, linear interpolation

Example: 99% uptime, 95% success, 60ms latency → 97 score

Critical Node Issues

1. **Lifecycle Manager BROKEN:** Uses `.query()/ .get()` but database is `better-sqlite3` with `.prepare()` API — will crash at runtime
2. **In-memory heartbeat stats:** Lost on worker restart, no persistence
3. **No watchdog for stale nodes:** No automatic deactivation for nodes that stop sending heartbeats

PART 9 — SCHEDULER AND RUNTIME LOOP REVIEW

Scheduler Inventory

Loop	File	Interval	Status
Job Dispatch	job_consumer.js	Continuous (pull-based)	■ Working — 5 concurrent workers
Demand Discovery	workload_discovery.js	60s	STUB — simulated market scanning
Demand Flywheel	demand_flywheel_engine.js	Per-completion	■ Real — 4 autonomous strategies
Demand Router Drain	demand_router.js	500ms	■ Real — bridges buffer → queue
Reputation Scoring	node_reputation.js	On heartbeat	■ Real — triggered per heartbeat
Epoch Finalization	—	—	■ MISSING — no cron or timer
Settlement Processing	—	—	■ MISSING — engine never called
Batch Creation	—	—	■ MISSING — no code exists
Heartbeat Watchdog	—	—	■ MISSING — no stale node detection

Crash Protection

Protection	Status
Job consumer retry	■■ Weak — immediately moves to DLQ instead of exponential backoff
Worker timeout	■ Missing — long-running jobs block worker
Graceful shutdown	■ Missing — no SIGTERM handler
DLQ processing	■ Exists — dead letter queue captures failed jobs
Queue backpressure	■ Exists — routes to external at >1M jobs

PART 10 — DATABASE INTEGRITY

Schema Verification

Table	Migration	Status	Purpose
nodes / registered_nodes	001_core.sql	■ Exists	Node registry
revenue_events_v2	002_revenue.sql	■ Exists	Revenue source of truth
epochs	004_epoch_distribution.sql	■ Exists	Epoch metadata
node_epoch_earnings	004_epoch_distribution.sql	■ Exists	Per-node earnings
claims	003_balances_claims_withdrawals.sql	■ Exists	Claim records
withdrawals	003_balances_claims_withdrawals.sql	■ Exists	Withdrawal queue
node_reputation	layer36_reputation.sql	■ Exists	Reputation scores
job_queue / job_queue_log	011_rpc_gateway.sql	■ Exists	Job tracking
execution_metrics	008_production_upgrade.sql	■ Exists	Performance data
ops_pricing	layer55_ops_rewards.sql	■ Exists	Operation prices
economic_ledger_entries	layer26_economic_ledger.sql	■ Exists	Double-entry ledger
economic_ledger_chain	layer26_economic_ledger.sql	■ Exists	Tamper-evident chain
payout_batches_v2	layer23_settlement_v2.sql	■ Exists	Settlement batches
payout_items_v2	layer23_settlement_v2.sql	■ Exists	Individual payouts
settlement_evm_txs	layer32_evm_settlement.sql	■ Exists	EVM tx tracking
feature_flags_v2	layer23_feature_flags.sql	■ Exists	Feature toggles

Database Technology

- **Current:** SQLite (better-sqlite3) with WAL mode, foreign keys, 5s busy timeout
- **Configured but unused:** PostgreSQL 15 in Docker
- **Migration system:** Sequential SQL files loaded at startup
- **Schema gaps:** None detected — all required tables exist

Critical Issue

SQLite is **NOT** production-ready for a DePIN network. Concurrent writes from multiple workers will cause WAL contention. The PostgreSQL container exists in docker-compose but the application hardcodes SQLite in `server.js`:

```
const db = new Database(process.env.SQLITE_PATH || "satelink.db");
```

PART 11 — MONITORING STACK

Monitoring Component Status

Component	Status	Details
/health endpoint	■ Working	HTTP 200 check in docker-compose
/metrics endpoint	■ Working	Prometheus format
/metrics/json endpoint	■■ Unverified	May exist via Prometheus exporter
Watchdog	■■ Health check only	Docker/K8s probe, NOT runtime monitoring
Pino Logger	■ Working	Structured JSON logging
Queue Health	■ Working	/health/queue → depth + pricing multiplier
Prometheus Config	■ Exists	infra/monitoring/prometheus/
Grafana Config	■ Exists	infra/monitoring/grafana/
AlertManager Config	■ Exists	infra/monitoring/alertmanager/
Scheduler Guard	■ Missing	No watchdog for scheduler health
Heartbeat Guard	■ Missing	No stale-node detection
Ledger Guard	■ Missing	No debit/credit balance verification loop
Memory Guard	■ Missing	No OOM protection
Watchdog Report Endpoint	■ Missing	No consolidated health report

What's Missing

- No runtime watchdog:** The `watchdog.js` is a Docker health probe, not a system state monitor
- No deadlock detection:** No monitoring for hung workers or stuck queues
- No alerting pipeline:** Prometheus/Grafana configs exist but aren't wired to the application
- No economic invariant checks:** No periodic verification that debits = credits in ledger

PART 12 — DOCKER INFRASTRUCTURE

Service Matrix

Service	Image	Health Check	Port	Status
satelink-core	Custom (Dockerfile)	HTTP /health	8080	■ Configured
satelink-gateway	Custom	None	8081	■■ No health check
satelink-node-worker	Custom	None	—	■■ No health check
database	postgres:15-alpine	pg_isready	5432	■ Configured
redis	redis:7-alpine	PING	6379	■ Configured
dashboard	Custom (Dockerfile.web)	None	3000	■■ No health check

Critical Docker Issues

Severity	Issue	File
CRITICAL	Hardcoded DB password: satelinkpassword	docker-compose.yml:64

Severity	Issue	File
CRITICAL	PostgreSQL provisioned but app uses SQLite	docker-compose.yml
HIGH	Dockerfile.backend runs as root	Dockerfile.backend
HIGH	Dockerfile.frontend runs as root, uses npm run dev	Dockerfile.frontend
HIGH	Redis has no persistence, no auth, no memory limits	docker-compose.yml
MEDIUM	No container security scanning in CI	.github/workflows/
MEDIUM	Deploy workflow missing env/secrets setup	deploy.yml
LOW	npm install instead of npm ci in Dockerfile.api	Dockerfile.api:10

Nginx Configuration

- **nginx.staging.conf** — Good reverse proxy with security headers (X-Frame-Options, XSS-Protection, Content-Type-Options)
- SSE properly configured with proxy_buffering off
- **Missing:** SSL not configured (commented out), listens on port 8000 (staging)

PART 13 — SECURITY REVIEW

CRITICAL Vulnerabilities

#	Vulnerability	File	Risk
1	Hardcoded Hardhat private keys as fallback	deploy_settlement.js:25,28	Key exposure if PRIVATE_KEY env not set
2	JWT_REFRESH_SECRET has hardcoded fallback 'satelink-fallback-refresh-token'	auth_middleware.js:25	Authentication bypass
3	PASSWORD_SALT cascades to JWT_SECRET to 'satelink_fallback_salt'	auth_v2.js:93	Weak password hashing
4	SQLite production fallback — warns but continues	db/index.js:319-329	Data integrity at scale
5	DB password in docker-compose	docker-compose.yml:64	Credential exposure in VCS

HIGH Vulnerabilities

#	Vulnerability	File	Risk
6	IP_HASH_SALT default 'satelink_default_salt_change_me'	admin_control_room_api.js:5	Privacy violation
7	Staging auth undefined env vars pass through	staging_auth.js:18	Credential bypass
8	Dev auth token generation in non-production envs	dev_auth_tokens.js:11-16	Unrestricted token gen

#	Vulnerability	File	Risk
9	No HTTPS enforcement or HSTS headers	middleware.js	MITM attacks
10	CORS allows all origins when empty	middleware.js:39-50	Cross-origin attacks

MEDIUM Vulnerabilities

#	Vulnerability	File	Risk
11	Rate limit IP spoofing (req.ip without proxy validation)	rate_limits.js:9-10	Rate limit bypass
12	Abuse firewall thresholds hardcoded (5000 ops)	abuse_firewall.js:10-16	Cannot adjust enforcement
13	Admin key timing attack (simple string comparison)	security.js:11-27	Key discovery
14	Token revocation store (Redis) not wired	auth_middleware.js:177	Cannot invalidate tokens
15	No Helmet HSTS configuration	middleware.js	No browser HTTPS enforcement

Positive Security Findings

- JWT_SECRET has no fallback — hard-fail if missing (32+ char enforced)
- HS256 algorithm enforced for JWT
- ECDSA signature verification on node pairing and claims
- RevenueDistributor uses SafeERC20 + ReentrancyGuard
- ClaimsWithdrawals uses Merkle proof verification (OpenZeppelin standard)
- Abuse firewall with per-wallet/per-op rate limiting
- System freeze after 10 failed admin key attempts
- Safe mode blocks all operations when activated
- No .env files committed to git

PART 14 — 72-HOUR TEST READINESS

Requirements vs Reality

Requirement	Status	Blocker
10 simulated nodes	■ PARTIAL	Node registration works, but lifecycle_manager DB API is broken
20 RPS workload generator	■ PARTIAL	Demand flywheel generates work, but all execution is MOCK
72-hour runtime	■ BLOCKED	No graceful shutdown, no worker recovery, no watchdog
Revenue tracking	■ READY	Operations engine records correctly

Requirement	Status	Blocker
Epoch closure	■ BLOCKED	closeEpoch() never called — no automated scheduler
Settlement	■ BLOCKED	Engine coded but not wired
Node heartbeat	■ READY	Works but stats in-memory
Monitoring	■ BLOCKED	No runtime watchdog, no alerting
Database stability	■ BLOCKED	SQLite WAL contention under concurrent load

Blockers for 72-Hour Test

- 1. **No epoch closure loop** — earnings never distributed
 - 2. **No settlement invocation** — payouts never processed
 - 3. **SQLite under load** — WAL contention with 10 nodes + 20 RPS
 - 4. **No crash recovery** — worker dies = jobs lost
 - 5. **No stale node detection** — fake nodes persist forever
 - 6. **lifecycle_manager broken** — DB API mismatch prevents node onboarding
- Verdict: 6 blocking issues must be resolved before the 72-hour test can begin.

PART 15 — REAL NETWORK LAUNCH READINESS

Launch Requirements Assessment

Requirement	Status	Gap
Real machine demand	■	All connectors are stubs
Real RPC traffic	■	Provider fallback adapter is mock
Real AI inference	■	AI source generates fake payloads
Real automation jobs	■	Cron source generates fake triggers
Real USDT settlement	■	Settlement engine not wired, EVM adapter not registered
Real node payouts	■	Pipeline terminates at withdrawal record
Production database	■	Using SQLite, not PostgreSQL
SSL/HTTPS	■	Not configured
Real wallet login	■	Test mode only
Container security	■	Root users, no scanning

Verdict: 10/10 requirements NOT MET. The system is not ready for real network launch.

PART 16 — INFRASTRUCTURE LAUNCH CHECKLIST

Pre-Launch Verification (18 Checks)

#	Category	Check	Status	Priority
1	Environment	All secrets set (JWT, salts, admin keys, private keys) — no fallbacks	■ FAIL	P0
2	Environment	CORS_ORIGINS explicitly configured (no wildcard)	■ FAIL	P0
3	Security	HTTPS enforced with valid SSL certificate	■ FAIL	P0
4	Security	All dev/test routes disabled (dev_seed, simulation_only, dev_auth_tokens)	■■ PARTIAL	P0
5	Security	JWT_REFRESH_SECRET set (no hardcoded fallback)	■ FAIL	P0
6	Database	PostgreSQL configured and migrated (not SQLite)	■ FAIL	P0
7	Economic	Epoch closure automated (scheduler or cron)	■ FAIL	P0
8	Economic	Settlement engine wired and processing batches	■ FAIL	P0
9	Economic	Ledger invariant check: total debits = total credits	■ FAIL	P0
10	Economic	50/30/20 split matches on-chain SplitEngine config	■■ PARTIAL	P1
11	Nodes	Heartbeat watchdog active (deactivates stale nodes)	■ FAIL	P1
12	Nodes	Node onboarding lifecycle working (lifecycle_manager fix)	■ FAIL	P1
13	API	Rate limiting validated (no IP spoofing)	■■ PARTIAL	P1
14	API	Health check returns real system state (not just 200 OK)	■■ PARTIAL	P1
15	Scheduler	Job consumer has retry + timeout (not instant DLQ)	■ FAIL	P1
16	Monitoring	Prometheus + Grafana dashboards connected	■■ EXISTS NOT WIRED	P1
17	Infrastructure	Redis has persistence, auth, and memory limits	■ FAIL	P1

#	Category	Check	Status	Priority
18	Infrastructure	Docker containers run as non-root with health checks	■ ■ PARTIAL	P2
19	Demand	At least 1 connector produces real market demand	■ FAIL	P1
20	Auth	Wallet login functional (not test mode)	■ FAIL	P1

Result: 0/20 checks PASS. 4/20 PARTIAL. 16/20 FAIL.

PART 17 — FIX BLUEPRINT

Stage 1 — Critical Fixes (Estimated: 20-30 hours)

Goal: Make the economic pipeline operational end-to-end

Task	File(s)	Hours	Description
1.1	src/scheduler/epoch_closure_job.js (NEW)	3	Create automated epoch closure loop (check epoch duration, call closeEpoch)
1.2	src/settlement/batch_creator.js (NEW)	4	Create withdrawal → payout_batches_v2 pipeline
1.3	server.js or worker.js	2	Wire SettlementEngine.processQueue() to a periodic timer
1.4	src/settlement/adapters/register.js	2	Register EvmAdapter + SimulatedAdapter at startup
1.5	.env.example	1	Document all 14 missing environment variables
1.6	auth_middleware.js:25	0.5	Remove JWT_REFRESH_SECRET fallback — require env var
1.7	auth_v2.js:93	0.5	Remove PASSWORD_SALT fallback — require env var
1.8	db/index.js:319	0.5	Hard-fail in production if DATABASE_URL not set
1.9	server.js	2	Add PostgreSQL connection support (use DATABASE_URL when set)
1.10	worker.js	2	Add graceful shutdown (SIGTERM handler, drain queue)

Stage 2 — Architecture Upgrades (Estimated: 20-25 hours)

Goal: Production-grade infrastructure

Task	File(s)	Hours	Description
2.1	lifecycle_manager.js	3	Fix DB API (.query()/get() → .prepare())
2.2	job_consumer.js	3	Add exponential backoff retry (3 attempts before DLQ)
2.3	job_consumer.js	2	Add job execution timeout (configurable, default 30s)
2.4	src/monitoring/watchdog_runtime.js (NEW)	4	Create runtime watchdog: stale node detection, queue depth alerts, memory guard
2.5	src/scheduler/ledger_integrity_job.js	2	Periodic ledger invariant check (debits = credits)
2.6	node_heartbeat.js	2	Persist heartbeat stats to DB (not in-memory)
2.7	docker-compose.yml	2	Remove hardcoded secrets, add env_file, Redis persistence + auth
2.8	Dockerfile.backend, Dockerfile.frontend	1	Add non-root USER, use npm ci, multi-stage builds
2.9	middleware.js	1	Add HSTS, enforce HTTPS redirect in production
2.10	security.js	1	Use timing-safe comparison for admin key

Stage 3 — Connector Activation (Estimated: 15-20 hours)

Goal: Real workload demand

Task	File(s)	Hours	Description
3.1	provider_fallback_adapter.js	4	Replace mock with real HTTP forwarding to external RPC providers
3.2	rpc_source.js	3	Connect to real RPC demand (or convert to realistic load generator)
3.3	src/core/config/rpc_providers.js	2	Configure real Infura/Alchemy API keys via environment
3.4	workload_discovery.js	3	Convert market scanner from stub to real market API integration
3.5	ai_source.js	3	Wire to real AI inference relay or realistic simulation
3.6	Feature flags	1	Create ENABLE_* flags per connector for safe rollout

Stage 4 — UI Repair (Estimated: 15-20 hours)

Goal: Functional wallet login and real dashboard data

Task	File(s)	Hours	Description
4.1	apps/dashboard/package.json	1	Install wagmi + @rainbow-me/rainbowkit (or web3modal)
4.2	apps/dashboard/src/app/login/page.tsx	4	Replace test buttons with WalletConnect + MetaMask UI
4.3	apps/dashboard/src/hooks/use-auth.tsx	2	Wire wallet connect → nonce challenge → signature → JWT
4.4	apps/dashboard/src/app/network/page.tsx	2	Replace mock AnimatedNodeCards with real node registry data
4.5	apps/dashboard/src/app/builders/page.tsx	1	Replace hardcoded "OPTIMAL" with real health data
4.6	apps/dashboard/src/app/node/claim/page.tsx	2	Wire claim/withdraw to working settlement pipeline
4.7	Add React error boundaries	2	Prevent white-screen crashes on component errors
4.8	Remove duplicate auth hook	1	Consolidate use-auth.tsx and AuthContext.tsx

Stage 5 — Final Launch Preparation (Estimated: 10-15 hours)

Goal: Pass all 20 launch checklist items

Task	File(s)	Hours	Description
5.1	SSL/nginx	2	Configure Certbot + HTTPS redirect
5.2	CI/CD	3	Add Docker build/push, security scanning (Trivy), health check verification
5.3	.github/workflows/deploy.yml	2	Add secrets injection, migration step, rollback strategy
5.4	PM2 config	1	Add clustering (2+ instances), log rotation, graceful shutdown timeout
5.5	Monitoring	2	Wire Prometheus → Grafana → AlertManager to application
5.6	Integration test suite	4	End-to-end test: register node → execute op → close epoch → claim → withdraw → settle

Total Estimated Effort

Stage	Hours	Priority
Stage 1 — Critical Fixes	20-30	IMMEDIATE
Stage 2 — Architecture Upgrades	20-25	WEEK 1
Stage 3 — Connector Activation	15-20	WEEK 2

Stage	Hours	Priority
Stage 4 — UI Repair	15-20	WEEK 2
Stage 5 — Final Launch Prep	10-15	WEEK 3
TOTAL	80-110	3 weeks

PART 18 — FINAL OUTPUT

System Architecture Summary

Satelink is a monorepo DePIN platform with:

- **Backend:** Node.js/Express with 200+ service files, Redis Streams job queue, SQLite database
- **Frontend:** Next.js 16 with 120+ pages, Tailwind/shadcn UI, SWR data fetching
- **Contracts:** 10 Solidity contracts (OpenZeppelin v4.9.6) — SplitEngine, RevenueDistributor, ClaimsWithdrawals, RevenueVault, EpochAnchor, NodeRegistryV2
- **Infrastructure:** Docker Compose (5 services), nginx reverse proxy, PM2 process manager, GitHub Actions CI/CD

Missing Implementations

1. Withdrawal → batch creation pipeline (no code exists)
2. Settlement engine invocation (coded but never called)
3. EVM adapter registration (coded but never registered)
4. Epoch closure automation (function exists, no scheduler)
5. Real wallet login (test mode only)
6. Real workload connectors (all stubs)
7. Real RPC provider forwarding (mock adapter)
8. PostgreSQL integration (SQLite used despite Postgres in Docker)

Security Risks (Prioritized)

1. **P0:** JWT_REFRESH_SECRET hardcoded fallback
2. **P0:** PASSWORD_SALT cascading fallback chain
3. **P0:** SQLite production fallback (should hard-fail)
4. **P0:** Hardcoded DB password in docker-compose
5. **P1:** CORS allows all origins when empty
6. **P1:** No HTTPS enforcement
7. **P1:** Dev routes accessible in non-production
8. **P1:** Admin key timing attack vulnerability

Disabled Systems

1. Settlement engine (coded, not wired)
2. EVM adapter (coded, not registered)
3. All 6 workload connectors (stubs only)
4. Provider fallback (mock responses)
5. Prometheus/Grafana/AlertManager (configs exist, not connected)
6. PostgreSQL (container runs, app uses SQLite)
7. Wallet login (test endpoint only)

1. Wire settlement engine to cron/timer
2. Create batch creation pipeline
3. Register EVM adapter with real configuration
4. Automate epoch closure
5. Fix lifecycle_manager DB API
6. Switch from SQLite to PostgreSQL
7. Remove all hardcoded secret fallbacks
8. Configure HTTPS/SSL
9. Implement wallet login
10. Add at least 1 real workload connector

1. Add runtime watchdog (stale node detection, queue alerts, memory guard)
2. Implement job consumer retry with exponential backoff
3. Persist heartbeat stats to database
4. Add React error boundaries in frontend
5. Wire Prometheus → Grafana → AlertManager
6. Add container security scanning in CI
7. Implement timing-safe admin key comparison
8. Add ledger integrity verification loop

```
████████████████████████████████████████████████████████████████████████████████
■ SATELINK DePIN NETWORK ■
■ PRODUCTION READINESS SCORECARD ■
████████████████████████████████████████████████████████████████████████████████
■ ■
■ Runtime Stability ..... 52% ████████████████████████████████████ ■
■ ✓ Server starts, routes work ■
■ ✓ Job queue functional ■
■ ✗ No crash recovery ■
■ ✗ No runtime watchdog ■
■ ✗ SQLite under concurrent load ■
■ ■
■ Economic Pipeline ..... 35% ████████████████████████████████████ ■
■ ✓ Revenue events recorded correctly ■
■ ✓ 50/30/20 split verified (3 locations) ■
■ ✓ Claims with ECDSA signature ■
■ ✗ Epoch closure never triggered ■
■ ✗ Settlement engine not wired ■
■ ✗ No on-chain USDT transfers ■
■ ■
■ Security Posture ..... 45% ████████████████████████████████████ ■
■ ✓ JWT auth hardened (no secret fallback) ■
■ ✓ Merkle-proof claims on-chain ■
■ ✓ SafeERC20 + ReentrancyGuard ■
■ ✗ Refresh token fallback ■
■ ✗ Password salt fallback ■
■ ✗ No HTTPS, open CORS ■
■ ■
■ Frontend Completeness ..... 60% ████████████████████████████████████ ■
■ ✓ 120+ pages built ■
■ ✓ SWR data fetching from API ■
■ ✓ Embedded wallet crypto implemented ■
■ ✗ No wallet login UI ■
```



END OF AUDIT REPORT

Prepared: 2026-03-16

Scope: Full repository audit — 18 parts

Files analyzed: 400+

Estimated fix effort: 80-110 engineering hours (3 weeks)

FINAL PRODUCTION READINESS SCORECARD

Category	Score	Visual	Rating
Runtime Stability	52%	[#####.....]	FAILING
Economic Pipeline	35%	[#####.....]	CRITICAL
Security Posture	45%	[#####.....]	AT RISK
Frontend Completeness	60%	[#####.....]	PARTIAL
Infrastructure Readiness	40%	[#####.....]	WEAK
Connector Readiness	15%	[###.....]	CRITICAL

OVERALL PRODUCTION READINESS	28%
72-HOUR TEST READINESS	22%
REAL LAUNCH READINESS	15%

Estimated fix effort: 80-110 engineering hours (3 weeks)
5-stage blueprint: Critical Fixes -> Architecture Upgrades -> Connector Activation -> UI Repair -> Launch Prep

END OF AUDIT REPORT